

```
[2]: import pandas as pd
df=pd.read_json("titanic.json")
df.head(10)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	None	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	None	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	None	S
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	None	Q
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	None	S
8	9	1	3	Johnson, Mrs. Oscar W	female	27.0	0	2	347742	11.1333	None	S

```
[4]: import pandas as pd
df=pd.read_excel("Students-Marksheet-Sample-Data.xlsx")
df.tail(10)
```

[4]:	Unnamed: 0	Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5	Unnamed: 6
27	NaN	120071	Alexander Davis	85	91	87	0.876667
28	NaN	120073	Mia Moore	93	95	92	0.933333
29	NaN	120082	Ethan Jackson	81	87	80	0.826667
30	NaN	120083	Isabella Brown	87	90	85	0.873333
31	NaN	120088	Noah Taylor	79	84	78	0.803333
32	NaN	120090	Abigail Johnson	90	92	89	0.903333
33	NaN	120093	Logan Martinez	82	87	83	0.84
34	NaN	120095	Avery Clark	88	91	87	0.886667
35	NaN	120098	Jacob Thompson	76	81	75	0.773333
36	NaN	120099	Samantha Harris	89	93	88	0.9

```
[39]: import pandas as pd
df=pd.read_json("titanic.json")
print(df)
df.head()

#check missing values
#isnull()
#used to identify missing values
print(df.isnull())
#count missing values of all fiels
print(df.isnull().sum())
#find rows missing age
print(df['Age'].isnull())
print(df.columns) #to identify the columns in the data

#fillna()
#replace missing values
#fill age with mean
mean_age=df['Age'].mean()
df['Age']=df['Age'].fillna(mean_age)
df['Cabin']=df['Cabin'].fillna('C83')
print(df.isnull().sum())

#Fill Embarked with most frequent value
#by using mode() means the most frequently occuring value in a column
mode_embarked=df['Embarked'].mode()[0]
df['Embarked']=df['Embarked'].fillna(mode_embarked)

#dropna()
#remove missing values
#remove rows
df.dropna()
#remove column
df.dropna(axis=1)
```

```

#groupby
#Average Fare Per Class
print(df.groupby('Pclass')['Fare'].mean())
#count survivors by gender
df.groupby('Sex')['Survived'].sum()
print(df.groupby('Sex')['Survived'].sum())

#multiple aggerations # for numerical we can use this
#agg() #function
print(df.groupby('Pclass').agg({
    'Age': ['mean', 'min', 'max'],
    'Fare': ['mean', 'sum']
}))

#transform()
#returns same number of rows.
#add class average Fare
df['Avg_Class_Fare']=df.groupby('Pclass')['Fare'].transform('mean')
print(df)
print(df.columns)

```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	
..	...	...	...	
886	887	0	2	
887	888	1	1	
888	889	0	3	
889	890	1	1	
890	891	0	3	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	None	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	None	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	None	S
..	...	...	...	...	...
886	0	211536	13.0000	None	S
887	0	112053	30.0000	B42	S
888	2	W./C. 6607	23.4500	None	S
889	0	111369	30.0000	C148	C
890	0	370376	7.7500	None	Q

[illegible]

888	False	False	False	False	False	False	True	False	False	False
889	False	False	False	False	False	False	False	False	False	False
890	False	False	False	False	False	False	False	False	False	False

	Fare	Cabin	Embarked
0	False	True	False
1	False	False	False
2	False	True	False
3	False	False	False
4	False	True	False
..	...	...	...
886	False	True	False
887	False	False	False
888	False	True	False
889	False	False	False
890	False	True	False

[891 rows x 12 columns]

PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

dtype: int64

0	False
1	False
2	False
3	False
4	False

```

...
886     False
887     False
888     True
889     False
890     False
Name: Age, Length: 891, dtype: bool
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
      'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
      dtype='object')
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin          0
Embarked       2
dtype: int64
Pclass
1      84.154687
2      20.662183
3      13.675550
Name: Fare, dtype: float64
Sex
female      233
male        109
Name: Survived, dtype: int64

```

	PassengerId	Survived	Pclass \
0	1	0	3
1	2	1	1
2	3	1	3
3	4	1	1
4	5	0	3
..	...	...	...
886	887	0	2
887	888	1	1
888	889	0	3
889	890	1	1
890	891	0	3

	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Avg_Class_Fare
0	1	0	A/5 21171	7.2500	C83	S	13.675550
1	1	0	PC 17599	71.2833	C85	C	84.154687
2	0	0	STON/O2. 3101282	7.9250	C83	S	13.675550

3	1	0	113803	53.1000	C123	S	84.154687
4	0	0	373450	8.0500	C83	S	13.675550
..	...	...	...	...	...	...	...
886	0	0	211536	13.0000	C83	S	20.662183
887	0	0	112053	30.0000	B42	S	84.154687
888	1	2	W./C. 6607	23.4500	C83	S	13.675550
889	0	0	111369	30.0000	C148	C	84.154687
890	0	0	370376	7.7500	C83	Q	13.675550

[891 rows x 13 columns]

```
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
      'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked', 'Avg_Class_Fare'],
      dtype='object')
```

```
[40]: #interpolate()
#estimate missing values
sample=pd.DataFrame({
    'Age':[20,25,None,35,40]
})
sample['Age']=sample['Age'].interpolate() #25 and 35 35-25=10 25+10/2=30 answer=30
print(sample)
```

```

      Age
0  20.0
1  25.0
2  30.0
3  35.0
4  40.0
```

```
[56]: import pandas as pd
students=pd.read_excel("Merge_Join_Concat_Dataset_50_Records.xlsx",sheet_name="students")
marks=pd.read_excel("Merge_Join_Concat_Dataset_50_Records.xlsx",sheet_name="marks")
contact=pd.read_excel("Merge_Join_Concat_Dataset_50_Records.xlsx",sheet_name="contact")
```

```

#1.INNER MERGE
#returns only matching student_IDs from both DataFrames
inner_df=pd.merge(      #it is used for common values alone like 50 data common
    students,
    marks,
    on="Student_ID",
    how="inner"
)
print(inner_df.head())
print(inner_df.shape)

#2.LEFT MERGE
left_df=pd.merge(      #it is used for common values alone like 50 data common
    students,
    marks,
    on="Student_ID",
    how="left"
)
print(left_df.tail())
print(left_df.shape)

#RIGHT MERGE
right_df=pd.merge(
    students,
    marks,
    on="Student_ID",
    how="right"
)
print(right_df.tail())
print(right_df.shape)

```

```
print(outer_df.shape)

#OUTER MERGE
outer_df=pd.merge(
    students,
    marks,
    on="Student_ID",
    how="outer"
)
print(outer_df.tail())
print(outer_df.shape)

#MERGE MULTIPLE TABLES
#combine students+marks+contact
student_complete=pd.merge(
    students,
    marks,
    on="Student_ID",
    how="left"
)
student_complete=pd.merge(
    student_complete,
    contact,
    on="Student_ID",
    how="left"
)
print(student_complete.head())
```

	Student_ID	Name	Department	Maths	Python
0	1001	Student_1	CSE	61	56
1	1002	Student_2	IT	62	57
2	1003	Student_3	ECE	63	58
3	1004	Student_4	EEE	64	59
4	1005	Student_5	MECH	65	60

(45, 5)

	Student_ID	Name	Department	Maths	Python
45	1046	Student_46	CSE	NaN	NaN
46	1047	Student_47	IT	NaN	NaN
47	1048	Student_48	ECE	NaN	NaN
48	1049	Student_49	EEE	NaN	NaN
49	1050	Student_50	MECH	NaN	NaN

(50, 5)

	Student_ID	Name	Department	Maths	Python
45	2001	NaN	NaN	75	80
46	2002	NaN	NaN	75	80
47	2003	NaN	NaN	75	80
48	2004	NaN	NaN	75	80
49	2005	NaN	NaN	75	80

(50, 5)

	Student_ID	Name	Department	Maths	Python
50	2001	NaN	NaN	75.0	80.0
51	2002	NaN	NaN	75.0	80.0
52	2003	NaN	NaN	75.0	80.0
53	2004	NaN	NaN	75.0	80.0
54	2005	NaN	NaN	75.0	80.0

(55, 5)

	Student_ID	Name	Department	Maths	Python	Email
0	1001	Student_1	CSE	61.0	56.0	student1@college.edu
1	1002	Student_2	IT	62.0	57.0	student2@college.edu
2	1003	Student_3	ECE	63.0	58.0	student3@college.edu
3	1004	Student_4	EEE	64.0	59.0	student4@college.edu
4	1005	Student_5	MECH	65.0	60.0	student5@college.edu

[62]: *#6. CONCAT ROW-WISE*
#split students into two parts

```

part1=students.iloc[:25]
part2=students.iloc[25:]
combined_rows=pd.concat([part1,part2])
print(combined_rows.shape)

```

(50, 3)

[59]: #7. CONCAT COLUMN-WISE

```
student_basic=students[['Student_ID','Name']]
student_marks=marks[['Python']]
combined_cols=pd.concat(
    [student_basic,student_marks],
    axis=1
)
print(combined_cols.head())
```

	Student_ID	Name	Python
0	1001	Student_1	56
1	1002	Student_2	57
2	1003	Student_3	58
3	1004	Student_4	59
4	1005	Student_5	60

[61]: #8 JOIN()

```
#join() works using index.
students_index=students.set_index("Student_ID")
contact_index=contact.set_index("Student_ID")
joined_df=students_index.join(contact_index)
print(joined_df.tail(10))
```

Student_ID	Name	Department	Email
1041	Student_41	CSE	student41@college.edu
1042	Student_42	IT	student42@college.edu
1043	Student_43	ECE	student43@college.edu
1044	Student_44	EEE	student44@college.edu
1045	Student_45	MECH	student45@college.edu
1046	Student_46	CSE	student46@college.edu
1047	Student_47	IT	student47@college.edu
1048	Student_48	ECE	student48@college.edu
1049	Student_49	EEE	student49@college.edu
1050	Student_50	MECH	student50@college.edu

File Edit View Run Kernel Settings Help

JupyterLab Python [conda env:base] * Anaconda Toolbox

```
[80]: #pivot table is used to summarize and analyze data by grouping rows and
#applyong aggreate function such as sum, mean, count, max, min, etc.

#Step 1 : Load Dataset
import pandas as pd
df=pd.read_csv("people-100.csv")
print(df.head())
print(df.columns)

#Step 2 : Create Age Column
df['Date of birth']=pd.to_datetime(df['Date of birth']) #convert the string to datetime
today=pd.Timestamp.today() #timestamp means current and today time
df['Age']=(
    (today-df['Date of birth']).dt.days // 365 #the difference between today and date of birth the days are been divided
)
print(df[['First Name','Sex','Age']].head())

#1. Basic pivot table
#average age by gender

pivot1=pd.pivot_table(
    df,
    values='Age',
    index='Sex' #seperate average and index value female and male
)
print(pivot1)

#2. Pivot Table with aggfunc="mean"
pivot2=pd.pivot_table(
    df,
    values='Age',
    index='Sex',
    aggfunc='mean'
)
```

#3. sum like aggerate function

```

pivot3=pd.pivot_table(
    df,
    values='Age',
    index='Sex',
    aggfunc='sum',
)
print(pivot3)

```

#4. Pivot count

```

pivot4=pd.pivot_table(
    df,
    values='Age',
    index='Sex',
    aggfunc='count',
)
print(pivot4)

```

#5. Multiple aggeration

```

pivot5=pd.pivot_table(
    df,
    values='Age',
    index='Sex',
    aggfunc=['median','max','min']
)
print(pivot5)

```

#6. pivot table with margins=True

#Adds overall average

```

pivot6=pd.pivot_table(
    df,
    values='Age',
    index='Sex',
    aggfunc='mean',

```

```

    margins=True
)
print(pivot6)

#7. Group by job tittle and gender
#average age by job tittle and gender
pivot7=pd.pivot_table(
    df,
    values='Age',
    index='Job Title',
    columns='Sex',
    aggfunc='mean',
)
print(pivot7)

#8. Count people by job and gender
pivot8=pd.pivot_table(
    df,
    values='User Id',
    index='Job Title',
    columns='Sex',
    aggfunc='count',
    fill_value=0
)
print(pivot8)
#nan fill value with 0

```

	Index	User Id	First Name	Last Name	Sex	\
0	1	88F7B33d2bcf9f5	Shelby	Terrell	Male	
1	2	f90cD3E76f1A9b9	Phillip	Summers	Female	
2	3	DbeAb8CcdfeFC2c	Kristine	Travis	Male	
3	4	A31Bee3c201ef58	Yesenia	Martinez	Male	
4	5	1bA7A3dc874da3c	Lori	Todd	Male	

	Email	Phone	Date of birth	\
0	elijah57@example.net	001-084-906-7849x73518	1945-10-26	
1	bethany14@example.com	214.112.6044x4913	1910-03-24	
2	bthompson@example.com	277.609.7938	1992-07-02	
3	kaitlinkaiser@example.com	584.094.6111	2017-08-03	
4	buchananmanuel@example.net	689-207-3558x7233	1938-12-01	

	Job Title
0	Games developer
1	Phytotherapist
2	Homeopath
3	Market researcher
4	Veterinary surgeon

```
Index(['Index', 'User Id', 'First Name', 'Last Name', 'Sex', 'Email', 'Phone',
      'Date of birth', 'Job Title'],
      dtype='object')
```

	First Name	Sex	Age
0	Shelby	Male	80
1	Phillip	Female	116
2	Kristine	Male	33
3	Yesenia	Male	8
4	Lori	Male	87

	Age
Sex	
Female	64.981132
Male	54.957447

	Age
Sex	
Female	64.981132
Male	54.957447

	Age
Sex	
Female	3444
Male	2583

```
Age
Sex
Female    53
Male      47
      median  max  min
      Age    Age  Age
Sex
Female    66.0  117   8
Male      62.0  117   5
      Age
Sex
Female    64.981132
Male      54.957447
All       60.270000
Sex                                Female    Male
Job Title
Accountant, chartered management    95.0    88.0
Accounting technician               17.0     NaN
Advertising account executive       18.0     NaN
Agricultural consultant             NaN    34.0
Air broker                         84.0     NaN
...
Travel agency manager              NaN   117.0
Veterinary surgeon                  NaN    87.0
Warden/ranger                      NaN    45.0
Warehouse manager                  NaN   101.0
Waste management officer            NaN    10.0

[91 rows x 2 columns]
Sex                                Female    Male
Job Title
Accountant, chartered management     1      1
Accounting technician                 1      0
Advertising account executive         1      0
Agricultural consultant               0      1
Air broker                           1      0
```

...	...	...
Travel agency manager	0	1
Veterinary surgeon	0	1
Warden/ranger	0	1
Warehouse manager	0	1
Waste management officer	0	1

[91 rows x 2 columns]

DATA CLEANING

```
[90]: import pandas as pd
df=pd.read_excel("Data_Cleaning_50Plus_Records.xlsx")
print(df.head())

#2. Renaming Columns
#Check Current Columns Names
print(df.columns)

#Rename Columns
df.rename(columns={
    'Student ID':'Student_ID',
    'Student Name ':'Student_Name',
    'Department' :'Department'
},inplace=True)
print(df.columns)

#3. Convert Data Types
#Check Data Types
print(df.dtypes)

#convert Students_ID to string
df['Student ID']=df['Student ID'].astype(str)
```

```
#convert age to integer
#first handle missing values.
df['Age']=df['Age'].fillna(0) #fill 0 in empty values
df['Age']=df['Age'].astype(int) #this convert to integer
```

```
#check:
print(df.dtypes)
```

#4. Detecting Duplicates Records

#Find duplicates

```
duplicates=df.duplicated()
```

```
print(duplicates) #if duplicate it return true and if not duplicate return false.
```

	Student ID	Student Name	Department	Age	Salary	Email
0	101	student1	IT	20.0	23075.0	student1@gmail.com
1	102	student2	ECE	18.0	35929.0	student2@gmail.com
2	103	STUDENT3	EEE	22.0	29718.0	student3@gmail.com
3	104	student4	MECH	20.0	21009.0	student4@gmail.com
4	105	Student5	CSE	19.0	47113.0	student5@gmail.com

```

Index(['Student ID', 'Student Name ', ' Department', 'Age', 'Salary', 'Email'], dtype='object')
Index(['Student_ID', 'Student_Name', ' Department', 'Age', 'Salary', 'Email'], dtype='object')
Student_ID      int64
Student_Name    object
Department      object
Age             float64
Salary          float64
Email           object
dtype: object
Student_ID      object
Student_Name    object
Department      object
Age             int32
Salary          float64
Email           object
    
```

```
dtype: object
```

```
0      False
```

```
1      False
```

```
2      False
```

```
3      False
```

```
4      False
```

```
5      False
```

```
6      False
```

```
7      False
```

```
8      False
```

```
9      False
```

```
10     False
```

```
11     False
```

```
12     False
```

```
13     False
```

```
14     False
```

```
15     False
```

```
16     False
```

```
17     False
```

```
18     False
```

```
19     False
```

```
20     False
```

```
21     False
```

```
22     False
```

```
23     False
```

```
24     False
```

```
25     False
```

```
26     False
```

```
27     False
```

```
28     False
```

```
29     False
```

```
30     False
```

```
31     False
```

```
32     False
```

```
33     False
```

```
34     False
```

         Code 

```
28     False
29     False
30     False
31     False
32     False
33     False
34     False
35     False
36     False
37     False
38     False
39     False
40     False
41     False
42     False
43     False
44     False
45     False
46     False
47     False
48     False
49     False
50     False
51     False
dtype: bool
```